

Tutorial 3 Week 5

Tutorial Team 26

A0327301U Chao Chien Hung
A0274850Y Yao Tianhao

September 8, 2026

Problem 1: Nova the Mars Rover

Write c = charged and l = low, and list policies in the order $[\pi(c), \pi(l)]$. The model is

State	Action	$P(c \mid s, a)$	$P(l \mid s, a)$	$R(s, a)$
Charged	Eco	0.95	0.05	7
Charged	Boost	0.70	0.30	10
Low	Eco	0.50	0.50	0
Low	Boost	0.10	0.90	2

Although the course default is reward on entry, this problem explicitly specifies rewards independent of the next state. We therefore use $R(s, a, s') = R(s, a)$ throughout Problem 1. For $0 \leq \gamma < 1$,

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \mid S_0 = s \right].$$

Part 1: Discounting

(a) Role of Discounting

Discounting controls the trade-off between immediate data collection and future battery availability. A reward k steps ahead receives weight γ^k . For bounded rewards, $\gamma < 1$ also makes the infinite sum finite:

$$|V^\pi(s)| \leq \frac{R_{\max}}{1 - \gamma}.$$

With $\gamma = 0.6$, a reward five steps ahead receives weight $0.6^5 = 0.07776$; with $\gamma = 0.9$, its weight is $0.9^5 = 0.59049$. Thus the former agent is more short-sighted, while the latter places greater value on preserving or restoring its battery. The corresponding effective horizons $1/(1 - \gamma)$ are approximately 2.5 and 10 steps.

However, greater patience need not change the optimal action at every state. As shown in part (b), **both $\gamma = 0.6$ and $\gamma = 0.9$ yield [boost, eco] in this particular MDP.**

(b) Optimal Policy as γ Varies

For any candidate value function, let $\Delta = V_c - V_l$. The advantages of eco over boost are

$$\begin{aligned} Q(c, \text{eco}) - Q(c, \text{boost}) &= -3 + 0.25\gamma\Delta, \\ Q(l, \text{eco}) - Q(l, \text{boost}) &= -2 + 0.40\gamma\Delta. \end{aligned}$$

Hence eco is greedy in the charged state exactly when $\gamma\Delta \geq 12$, and in the low state exactly when $\gamma\Delta \geq 5$.

For a fixed policy with charged-state transition probabilities p_c, p_l and rewards r_c, r_l , subtracting its two Bellman equations gives

$$\Delta^\pi = (r_c - r_l) + \gamma(p_c - p_l)\Delta^\pi, \quad \Delta^\pi = \frac{r_c - r_l}{1 - \gamma(p_c - p_l)}.$$

The three possible optimal deterministic policies are obtained as follows.

- [boost, boost]: $\Delta = 8/(1 - 0.6\gamma)$. Both boost actions are greedy when $\gamma\Delta \leq 5$, giving $\gamma \leq 5/11$.
- [boost, eco]: $\Delta = 10/(1 - 0.2\gamma)$. The policy is greedy when $5 \leq \gamma\Delta \leq 12$, giving $5/11 \leq \gamma \leq 30/31$.
- [eco, eco]: $\Delta = 7/(1 - 0.45\gamma)$. Both eco actions are greedy when $\gamma\Delta \geq 12$, giving $\gamma \geq 30/31$.

Thus, away from the two tie points,

Discount factor	Unique optimal policy	Example
$0 \leq \gamma < 5/11$	[boost, boost]	$\gamma = 0.2$
$5/11 < \gamma < 30/31$	[boost, eco]	$\gamma = 0.8$
$30/31 < \gamma < 1$	[eco, eco]	$\gamma = 0.99$

At $\gamma = 5/11 \approx 0.45455$, boost is optimal when charged, while both low-state actions tie. At $\gamma = 30/31 \approx 0.96774$, eco is optimal when low, while both charged-state actions tie. Either adjacent deterministic policy, or randomization between the tied actions, is optimal at the respective boundary.

The policy [eco, boost] would require both $\gamma\Delta \geq 12$ and $\gamma\Delta \leq 5$, which is impossible. At $\gamma = 1$, the undiscounted total-reward objective diverges here; the table describes $\gamma < 1$, not a finite-valued solution at $\gamma = 1$.

(c) A Minimal-Entry Modification

Changing only the common discount factor cannot make [eco, boost] optimal, by the incompatible inequalities above. Instead, set $\gamma = 0.8$ and change **only one reward entry**:

$R(l, \text{eco}) = -3.1 \quad \text{instead of } 0.$

Keep all transition probabilities and the other three rewards unchanged. This can represent an operating penalty for using eco mode with a low battery.

Under the proposed policy $\pi = [\text{eco}, \text{boost}]$, the evaluation equations are unaffected by this change, since the policy does not choose eco when low:

$$\begin{aligned} V_c &= 7 + 0.8(0.95V_c + 0.05V_l), \\ V_l &= 2 + 0.8(0.10V_c + 0.90V_l). \end{aligned}$$

Solving gives $V^\pi = [31.875, 16.25]$. The one-step action values with continuation V^π are

$$\begin{aligned} Q(c, \text{eco}) &= 31.875, & Q(c, \text{boost}) &= 31.75, \\ Q(l, \text{eco}) &= -3.1 + 0.8(0.5 \cdot 31.875 + 0.5 \cdot 16.25) = 16.15, & Q(l, \text{boost}) &= 16.25. \end{aligned}$$

Both chosen actions are strictly better, so this policy satisfies Bellman optimality and is uniquely optimal.

For precision, if the modified reward is $R(l, \text{eco}) = \rho$, then its action value is $\rho + 19.25$. Thus $\rho \leq -3$ is necessary and sufficient for this policy to be optimal at $\gamma = 0.8$ when only this entry changes. The boundary $\rho = -3$ gives a tie; any $\rho < -3$ gives strict preference. The example changes the minimum possible number of model entries (one), and is close to the boundary for this particular reward change. It does not claim a globally smallest numerical change across rewards and probabilities.

(d) Practical Meaning and the Case $\gamma = 1$

The discount factor is the relative weight assigned to reward one time step later. It may represent time preference or, in an equivalent model with independent termination, the probability of continuing for another step. It is not itself one of the rover's state-transition probabilities.

At $\gamma = 1$, future rewards receive full weight. In this continuing MDP, all four stationary deterministic policies have positive long-run average reward, so their expected total rewards are infinite. Infinite values do not provide a useful ranking, and the Bellman operator is no longer guaranteed to be a contraction. One can instead use $\gamma < 1$, a finite horizon, termination with suitable finite-return conditions, or an average-reward objective. Thus $\gamma = 1$ is not inherently invalid, but requires a suitable problem formulation.

Part 2: Optimal Policy ($\gamma = 0.8$)

We now return to the **original, unmodified MDP**. Define

$$Q^{(k+1)}(s, a) = R(s, a) + 0.8 \sum_{s'} P(s' | s, a) V^{(k)}(s'), \quad V^{(0)} = [0, 0].$$

(a) First Q-Value Update

Since all continuation values are zero,

$Q^{(1)}(c, \text{eco}) = 7, \quad Q^{(1)}(c, \text{boost}) = 10, \quad Q^{(1)}(l, \text{eco}) = 0, \quad Q^{(1)}(l, \text{boost}) = 2.$
--

(b) Next Value Function

Taking the maximum over actions at each state gives

$$V^{(1)} = [\max(7, 10), \max(0, 2)] = [10, 2].$$

(c) Value Iteration versus Policy Iteration

Value iteration. Starting from $V^{(1)} = [10, 2]$,

$$\begin{aligned} Q^{(2)}(c, \text{eco}) &= 7 + 0.8(0.95 \cdot 10 + 0.05 \cdot 2) = 14.68, \\ Q^{(2)}(c, \text{boost}) &= 10 + 0.8(0.70 \cdot 10 + 0.30 \cdot 2) = 16.08, \\ Q^{(2)}(l, \text{eco}) &= 0 + 0.8(0.50 \cdot 10 + 0.50 \cdot 2) = 4.80, \\ Q^{(2)}(l, \text{boost}) &= 2 + 0.8(0.10 \cdot 10 + 0.90 \cdot 2) = 4.24. \end{aligned}$$

Therefore,

$$V_{\text{VI}}^{(2)} = [16.08, 4.80].$$

Policy iteration. The greedy policy with respect to $V^{(1)}$ uses the same one-step lookahead values above, not just the immediate rewards:

$$\pi^{(1)} = [\text{boost}, \text{eco}].$$

Its exact evaluation solves

$$\begin{aligned} V_c &= 10 + 0.8(0.7V_c + 0.3V_l), \\ V_l &= 0.8(0.5V_c + 0.5V_l), \end{aligned}$$

or equivalently,

$$\begin{bmatrix} 0.44 & -0.24 \\ -0.40 & 0.60 \end{bmatrix} \begin{bmatrix} V_c \\ V_l \end{bmatrix} = \begin{bmatrix} 10 \\ 0 \end{bmatrix}.$$

The second equation gives $V_l = \frac{2}{3}V_c$. Substitution yields

$$V_{\text{PI}}^{(2)} = V^{\pi^{(1)}} = \left[\frac{250}{7}, \frac{500}{21} \right] \approx [35.71429, 23.80952].$$

VI performs one Bellman optimality backup, whereas exact policy evaluation accounts for indefinitely following the selected policy. Their updates are therefore not expected to have the same values.

(d) Optimal Policy

Checking both actions using the exactly evaluated values gives

State	$Q(s, \text{eco})$	$Q(s, \text{boost})$
Charged	$737/21 \approx 35.09524$	$250/7 \approx 35.71429$
Low	$500/21 \approx 23.80952$	22

The policy remains greedy with respect to its own value function, so

$$\pi^*(c) = \text{boost}, \quad \pi^*(l) = \text{eco}, \quad V^* = \left[\frac{250}{7}, \frac{500}{21} \right].$$

Problem 2: Aster’s Rescue Route—Comparing Active RL Methods

Here $\gamma = 0.8$. We use the reward on each observed transition, with $U(T) = 0$ and all terminal action values zero. In particular, the reward 5 is received on entering T , not as an additional terminal-state utility.

(a) Active ADP

Action a still has four observations, all leading to s_1 . After the new transition under b , action b has five observations: three to T and two to s_1 . The maximum-likelihood estimates, without smoothing, are

Action	$\hat{P}(s_1 \mid s_0, \cdot)$	$\hat{P}(T \mid s_0, \cdot)$
a	1	0
b	$2/5$	$3/5$

Using the specified current estimate $U(s_1) = 4$,

$$\begin{aligned} \hat{Q}(s_0, a) &= 1(0 + 0.8 \cdot 4) = 3.2, \\ \hat{Q}(s_0, b) &= \frac{3}{5}(6 + 0.8 \cdot 0) + \frac{2}{5}(-2 + 0.8 \cdot 4) = 3.6 + 0.48 = 4.08. \end{aligned}$$

Thus the greedy action is $\boxed{b}$ because $4.08 > 3.2$. These are model-based one-step lookahead values using the given utilities; we do not replace $U(s_1) = 4$ with the later observed reward 5 in this subpart.

(b) Monte Carlo Control

The new episode’s discounted return following (s_0, b) is

$$G_0 = -2 + 0.8(5) = 2.$$

The sample-average estimate over all five complete returns is

$$Q_{\text{MC}}(s_0, b) = \frac{6 + 6 + 6 + 2 + 2}{5} = \frac{22}{5} = 4.4.$$

Equivalently, the previous mean was 5, so the incremental update is $5 + \frac{1}{5}(2 - 5) = 4.4$. No extra terminal reward is added.

(c) SARSA and Q-Learning

The agents start separately from $Q(s_0, b) = 2$, use $\alpha = 0.5$, and observe reward -2 followed by state s_1 . The next action actually selected is d .

SARSA uses the value of that selected action:

$$\begin{aligned} Q_{\text{new}}(s_0, b) &= Q(s_0, b) + \alpha[r + \gamma Q(s_1, d) - Q(s_0, b)] \\ &= 2 + 0.5[-2 + 0.8(1) - 2] = \boxed{0.4}. \end{aligned}$$

Its target is -1.2 . This is an on-policy update because it follows the behavior policy's next action.

Q-learning instead uses the largest next-state action value:

$$\begin{aligned} Q_{\text{new}}(s_0, b) &= Q(s_0, b) + \alpha[r + \gamma \max_{a'} Q(s_1, a') - Q(s_0, b)] \\ &= 2 + 0.5[-2 + 0.8 \max(4, 1) - 2] = \boxed{1.6}. \end{aligned}$$

Its target is 1.2. It is off-policy because the target uses the greedy action c even though the behavior policy selects d .

These are the updates for the first transition using the supplied Q-values. If the later transition $(s_1, d, 5, T)$ is also processed, both agents update $Q(s_1, d)$ to $1 + 0.5(5 - 1) = 3$; ordinary one-step updates do not retroactively repeat the first-transition update.

(d) If the Behavior Policy Selects c Instead

SARSA now bootstraps from $Q(s_1, c) = 4$, giving

$$Q_{\text{new}}^{\text{SARSA}}(s_0, b) = 2 + 0.5[-2 + 0.8(4) - 2] = \boxed{1.6}.$$

The Q-learning update remains $\boxed{1.6}$ because its maximum over next-state actions is unchanged. The updates coincide in this case because the selected action c is currently greedy.

(e) Comparison of the Four Methods

Method	What it learns	How experience improves the policy
Active ADP	A transition/reward model and values computed from it.	Updates model estimates, plans using expected Bellman backups, and chooses actions using the resulting values, with exploration.
Monte Carlo control	Action values from sampled complete returns; no model.	Averages returns after episodes, then improves the policy toward actions with larger estimated values while retaining exploration.
SARSA	Action values for the behavior policy; no model.	Updates after each transition using $r + \gamma Q(s', a')$ for the action actually selected, then improves an exploratory policy.
Q-learning	Optimal action values as its learning target; no model.	Updates after each transition using $r + \gamma \max_{a'} Q(s', a')$; the behavior policy may explore independently of the greedy target.

ADP uses a learned model to average over possible next states. Monte Carlo uses the sampled return without bootstrapping. SARSA and Q-learning both bootstrap from current value estimates, so they can update before the episode ends. The sample-average Monte Carlo calculation here uses returns under the observed continuation behavior; off-policy Monte Carlo control would need an appropriate correction if evaluating a different policy.

(f) Why Exploration Is Necessary

Without exploration, early inaccurate estimates can make the agent repeatedly choose a suboptimal action and never discover a better one. In particular:

- ADP needs observations of alternative actions to estimate their transition probabilities and rewards.
- Monte Carlo needs complete returns from different state–action pairs.
- SARSA needs experience of alternatives to improve its behavior policy.
- Q-learning still needs actual samples of each relevant state–action pair; taking a maximum in its target does not supply missing experience.

An ϵ -greedy policy is one way to balance exploration with exploitation. For asymptotic tabular guarantees, merely exploring once is not enough: suitable coverage and learning-rate conditions are needed. On-policy control commonly uses exploration that becomes greedy in the limit while still visiting relevant state–action pairs infinitely often.

Problem 3 (Optional): Reinforcement Learning in the Real World

(a) Modeling Human Behavior and Decision Making

An RL model can represent a person's situation as a state, a choice as an action, and feedback as reward. A prediction-error update compares an observed outcome with its expectation: unexpectedly good outcomes increase estimated value, whereas disappointing outcomes decrease it. This links learning from feedback to changes in future choices [?].

Human decisions need not follow a purely model-free rule. In a two-stage choice experiment, Daw et al. found evidence for both model-free reinforcement and model-based use of transition structure. Their striatal signals also reflected both kinds of prediction [?]. Thus a useful modeling approach is to combine cached action values with forward planning, and test the resulting predictions against observed choices rather than assume one algorithm explains all behavior.

(b) Connection to Neuroscience

Schultz, Dayan, and Montague related dopamine-neuron responses to reward prediction errors: an unexpected reward can elicit increased activity; after learning, the response can shift to a predictive cue; omission of an expected reward can produce a decrease at the expected time [?]. This resembles the temporal-difference error

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t).$$

Biologically grounded outcomes such as food provide reinforcement, while initially neutral cues can acquire predictive value. Reward and prediction error are different: an expected reward can remain valuable even when it produces little prediction error [?].

The influence is bidirectional: TD theory helped interpret neural data rather than being invented after these findings [?]. Evidence of interacting model-based and model-free processes [?] also suggests an AI design lesson: combine learned values with planning. This is an inference, not proof that such hybrid agents duplicate the brain.

Generative AI Acknowledgement

OpenAI Codex (GPT-5) was used as a discussion and refinement aid while preparing this document. Its main uses were to provide high-level guidance and hints during discussion, and to refine the wording and $\text{\LaTeX}$ formatting. The authors reviewed and verified the final solutions and take responsibility for their correctness. No search engine or other generative AI model was used in preparing this document.